

INSTITUTO PROFESIONAL SAN SEBASTIAN

APLICACIÓN MÓVIL DE LISTA DE TAREAS CON EXPO Y REACT NATIVE

https://github.com/calvareza/mis_tareas_expo

Asignatura: IF206IINF Desarrollo de Aplicaciones Móviles

Profesor: Boris Marcelo Belmar Muñoz

Integrantes:
Neervison Bonny Laguna Sánchez
Carlos Oriel Álvarez Arévalo

24 de Octubre de 2026

ACREDITACIÓN 2025
Construimos juntos una formación de calidad

1. Introducción y descripción del proyecto

El problema a resolver es un clásico del desarrollo móvil: las personas necesitan registrar sus pendientes de forma rápida y confiable, sin perderlos ni olvidar qué debían hacer. La solución construida es «Mis Tareas», una aplicación móvil de lista de tareas (To-Do List) que permite registrar y visualizar tareas pendientes desde el teléfono.

Cada tarea tiene un título (obligatorio) y una descripción (opcional), y la aplicación cumple los requisitos del sistema:

- Agregar tareas: formulario con título y descripción; al guardar, el formulario se limpia solo.
- Editar tareas: el formulario se carga con los datos de la tarea y permite actualizarlos.
- Eliminar tareas: botón por tarea, con confirmación previa para evitar borrados por error.
- Marcar tareas como completadas: checkbox que tacha la tarea y actualiza el contador.
- Visualizar la lista completa, con un resumen de tareas pendientes y completadas.

El proyecto se ejecutó y probó de forma real: se verificaron las versiones de Node.js y Expo, se comprobó el código con TypeScript y ESLint sin errores y se empaquetó la aplicación web con Metro (416 módulos). Los resultados se resumen en la sección de funcionalidades de este documento.

2. Framework elegido y por qué

Para este proyecto se eligió Expo con React Native y TypeScript. No es una elección arbitraria: un solo código sirve para Android, iOS y Web; los componentes son nativos reales; la app se prueba al instante en el teléfono real con Expo Go, sin compilar; y TypeScript detecta los errores de tipos antes de ejecutar. La siguiente tabla muestra cómo se implementó cada pieza de la interfaz:

Pieza de la interfaz	**Implementación en este proyecto**
Encabezado de la pantalla	Componente propio Encabezado dentro de SafeAreaView
Área de contenido	View + KeyboardAvoidingView; el desplazamiento lo aporta la lista
Lista de elementos	FlatList (lista virtualizada)
Elemento de lista	Componente propio TareaItem (una

	tarjeta por tarea)
Campos de texto	TextInput (título y descripción, esta última con multiline)
Botones	Pressable con estilos propios (Guardar, Actualizar, Editar, Eliminar, Cancelar)
Casilla de verificación	Checkbox personalizado con Pressable (círculo que se rellena con ✓)
Iconos	Iconos Ionicons de @expo/vector-icons
Pantalla (organización)	src/pantallas/PantallaTareas.tsx + componentes en src/componentes/ + StyleSheet por componente
Tema visual	Tokens en src/tema/ (colores, espaciados, radios y tipografía centralizados)

3. Tecnologías utilizadas y por qué se usa cada una

No se eligió «un solo programa»: la aplicación combina varias tecnologías, cada una con un rol claro y necesario. Entender ese rol es clave para poder defender el proyecto:

Tecnología	Qué es	Por qué se usa en este proyecto
Expo	Framework y plataforma sobre React Native: herramientas, librerías y servicios para crear, probar y publicar apps.	Permite desarrollar y probar en Android, iOS y Web con un solo código, sin configurar Android Studio ni Xcode de forma compleja. Con Expo Go la app se prueba en el teléfono real escaneando un código QR.
React Native	Framework que crea interfaces con componentes nativos reales.	La app se siente y rinde como nativa en Android e iOS manteniendo una sola base de código en TypeScript. Un botón es un botón real del sistema, no una página web disfrazada.
React	Librería de interfaces basada en componentes,	La pantalla se arma con componentes reutilizables (Encabezado,



	props, estado y hooks.	FormularioTarea, TareaItem) y la interfaz se actualiza sola cuando cambia el estado.
TypeScript	JavaScript con tipos; es el lenguaje del proyecto.	Detecta errores antes de correr la app: si a una Tarea le falta el titulo, el editor y tsc lo avisan. Código más seguro y fácil de mantener.
Node.js + npm/npx	Entorno que ejecuta las herramientas JavaScript y su gestor de paquetes.	No es la app: es solo el entorno de trabajo. Con él se crea el proyecto (create-expo-app), se instalan dependencias (npm install) y se levanta el servidor de desarrollo (npx expo start).
Metro	El empaquetador (bundler) de React Native/Expo.	Toma todos los archivos .tsx y los convierte en un paquete que el teléfono o el navegador pueden ejecutar; recarga la app al guardar cambios (Fast Refresh). Viene integrado en Expo.
Expo Go	App gratuita para Android/iOS que ejecuta proyectos Expo en desarrollo.	Permite probar la app en el teléfono real en segundos, sin compilar ni instalar APK: se escanea el QR que muestra npx expo start.
@expo/vector-icons (Ionicons)	Librería oficial de Expo con iconos vectoriales profesionales (estilo outline).	Da el lenguaje visual de la app —portapapeles, sol/luna, lápiz, basurero, gráfico, trofeo— sin archivos de imagen: cada icono se escala y se colorea con los tokens del tema. Viene incluida en Expo Go.
react-native-svg	Dibujo de gráficos SVG vectoriales dentro de React Native.	Con ella se dibuja la mascota como formas vectoriales que cambian según el estado (ojos, boca, brazos), sin imágenes y sin perder nitidez. Viene incluida en Expo Go (SDK 58).

En una frase: Node.js sostiene las herramientas, Expo organiza el proyecto y lo conecta con el teléfono, React Native + React dibujan la interfaz nativa y TypeScript cuida que el código no tenga errores de forma.

4. Preparación del entorno e instalación paso a paso

Requisitos: Node.js 20 o superior y la app Expo Go en el teléfono (Android o iOS). El proyecto se creó desde cero siguiendo estos pasos:

```
node -v    # 1) Verificar Node.js (resultado en este equipo: v24.22.1)
npx create-expo-app tareas-expo --template blank-typescript    # 2) Crear el proyecto
cd tareas-expo    # 3) Entrar a la carpeta
npm install    # 4) Instalar dependencias (solo la primera vez)
npx expo start    # 5) Levantar la aplicación
```

Al ejecutar npx expo start aparece un código QR: en el teléfono se abre Expo Go y se escanea el QR para ver la app real; en el navegador se presiona la tecla w para abrir la versión web; con un emulador Android abierto se presiona la tecla a. Para detener el servidor se usa Ctrl + C en la terminal. Como verificación opcional de tipos, npx tsc --noEmit debe terminar sin errores (así ocurrió en este proyecto).

5. Conceptos fundamentales de Expo y React Native

En este apartado se explican los conceptos fundamentales del framework seleccionado —Expo y React Native—: su arquitectura, sus componentes clave y su terminología, siempre con ejemplos del propio proyecto.

5.1 Arquitectura

Expo es una capa sobre React Native que entrega el proyecto listo (plantillas), librerías oficiales, el servidor de desarrollo y Expo Go para probar sin compilar. En React Native el código se escribe en TypeScript/JSX y se traduce a vistas nativas reales de Android e iOS: se escriben los componentes una vez y se ven nativos en ambas plataformas. Metro es el empaquetador que convierte los archivos .tsx en un paquete ejecutable por el teléfono o el navegador, con recarga al guardar (Fast Refresh). La comunicación entre el código JavaScript y la parte nativa se realiza mediante el bridge clásico y, en la nueva arquitectura (Fabric/TurboModules), de forma directa y más rápida.

5.2 Componentes clave y terminología

Concepto	Explicación y ejemplo en el proyecto
Componente	**Pieza reutilizable de interfaz escrita como función que devuelve JSX. Ejemplo: Tarealtem se dibuja una vez por cada tarea de la lista.**
JSX	**Sintaxis que mezcla etiquetas y JavaScript: {tarea.titulo} inserta un dato dentro de la vista.**
Props	**Datos que un componente padre entrega a un hijo; son de solo lectura. Ejemplo: Encabezado recibe totalPendientes y totalCompletadas.**
Estado (state)	**Datos que cambian con el tiempo (useState). Al cambiar, React vuelve a dibujar solo lo necesario. Ejemplo: la lista de tareas en useTareas.**
Hooks	**Funciones de React que dan capacidades a los componentes: useState (estado), useEffect (cargar el formulario al editar), useCallback/useMemo (optimizar) y hooks propios como useTareas.**
StyleSheet	**Estilos como objetos JavaScript por componente (equivalente a CSS, propiedades en camelCase); Flexbox ordena el layout.**
FlatList	**Lista virtualizada: solo dibuja los elementos visibles y recicla los demás, por lo que rinde bien aunque la lista crezca.**
Expo Go	**Contenedor oficial para ejecutar la app en desarrollo escaneando el código QR, sin compilar.**

6. Patrones de diseño aplicados

En este apartado se identifican los patrones de diseño centrales empleados por el framework, con ejemplos concretos de cómo se implementan en este proyecto y el archivo exacto donde se ven.

Patrón	Qué significa	Dónde se ve en el código
Arquitectura basada en	La UI se compone de	App.tsx compone

componentes	piezas pequeñas, reutilizables y combinables.	Encabezado + FormularioTarea + TareaItem.
Interfaz declarativa	Se describe qué se debe ver según el estado, no los pasos para dibujarlo.	TareaItem.tsx: si tarea.completada es true, el texto se tacha; no hay código que «tache a mano».
Contenedor / Presentacional	Un componente concentra la lógica y los demás solo presentan datos.	Contenedor: App.tsx. Presentacionales: los tres de src/componentes/.
Separación de responsabilidades	Cada archivo hace una sola cosa.	src/modelos/ (forma), src/hooks/ (lógica), src/componentes/ (vista), src/tema/ (estilo).
Hook personalizado	La lógica de negocio se encapsula en una función use… fuera de la pantalla.	src/hooks/useTareas.ts: agregar, actualizar, eliminar, completar y contadores.
Estado inmutable	Nunca se modifica el arreglo original; cada cambio crea uno nuevo para que React detecte el cambio.	useTareas.ts: [...previas, nueva], map(...) y filter(...).
Observador (reactivo)	Quien usa el estado queda «suscrito»: si el estado cambia, la vista se actualiza sola.	Al agregar una tarea, FlatList se re-renderiza sin manipular la lista a mano.
Componente controlado	El formulario no guarda su propio valor: lo guarda React (value + onChangeText).	FormularioTarea.tsx con TextInput para título y descripción.
Lista virtualizada (patrón List)	Solo se dibujan los elementos visibles; los demás se reciclan.	FlatList en App.tsx con keyExtractor por id.
Fuente única de verdad (diseño)	Colores y medidas viven en un solo archivo y todos lo importan.	src/tema/colores.ts

7. Diseño de la interfaz y principios aplicados

En este apartado se explica cómo se aplicaron los principios básicos de diseño de UI —jerarquía visual, balance, contraste, ritmo y proporción— indicando cómo se aplicó cada uno con valores concretos.

Principio	Cómo se aplicó en «Mis Tareas»
Jerarquía visual	Los iconos Ionicons de cada sección refuerzan esa jerarquía: cada bloque se reconoce por su símbolo y su palabra.Título «Mis tareas» en 28 px negrita, blanco sobre bloque azul (#1D4ED8) → tarjeta del formulario → título de lista en 18 px → tareas. Lo importante se ve primero y más grande.
Contraste	Texto #111827 sobre fondos claros, blanco sobre primario #1D4ED8 y descripciones en gris oscuro legible #4B5563 (nivel AA). El rojo #B91C1C se reserva para eliminar y el verde #15803D para completado.
Ritmo	Todos los márgenes y rellenos usan la escala 8 / 16 / 24 / 32 definida en src/tema/espaciado.ts; ninguna medida es al azar. Radios de borde consistentes: 12 y 16.Todos los márgenes y rellenos usan la escala 8 / 16 / 24 / 32 definida en src/tema/colores.ts; ninguna medida es al azar. Radios de borde consistentes: 12 y 16.
Balance	En el encabezado, la columna de textos (saludo, título, resumen) se equilibra con la mascota a la derecha.En cada tarjeta de tarea, el checkbox circular a la izquierda equilibra las acciones Editar/Eliminar a la derecha; el contenido (título y descripción) ocupa el centro.
Proporción	El formulario es una tarjeta acotada y la lista ocupa el resto de la pantalla (flex: 1 en la pantalla), porque el contenido principal es la lista de tareas.El formulario es una tarjeta acotada y la lista ocupa el resto de la pantalla (flex: 1 en App.tsx), porque el contenido principal es la lista de tareas.

8. Componentes de UI implementados

En este apartado se presentan los componentes de UI comunes implementados en la aplicación (botones, menús, formularios, listas y encabezados) y cómo se usa cada uno, de manera consistente. En una app de una sola pantalla, el rol de



menú y navegación lo cumplen el encabezado y los chips de filtro, que cambian entre las tres vistas de la lista (todas, pendientes y completadas).

Componente	Uso en la app
SafeAreaView	Encabezado seguro: el contenido no queda bajo el notch o la barra del sistema.
Encabezado (propio)	Saludo según la hora con su icono, título de la app, resumen de pendientes/completadas y la mascota que reacciona al avance.título de la app y resumen de pendientes/completadas.
View	Contenedor básico (equivalente a un div).
Text	Todos los textos: títulos, descripciones, botones y mensajes.
TextInput	Formulario: título en una línea y descripción multilínea.
Pressable	Botones Guardar/Actualizar, Cancelar, Editar y Eliminar, y el checkbox.
FlatList	Lista de tareas virtualizada, con estado vacío.
Tarealtem (propio)	Tarjeta de cada tarea.
Checkbox personalizado	Círculo que se rellena en verde con ✓ al completar.
KeyboardAvoidingView	El formulario no queda tapado por el teclado al escribir.
Alert	Confirmación antes de eliminar una tarea.
StatusBar (expo-status-bar)	Barra de estado del sistema coherente con el diseño.
Ionicons (@expo/vector-icons)	Iconos profesionales de toda la interfaz: saludo (sol/luna), portapapeles, gráfico de progreso, trofeo, lápiz, basurero, guardar, cancelar y los chips de filtro.
BarraProgreso (propio)	Barra animada del porcentaje completado, con el texto «X de Y tareas listas (Z%)» y un mensaje dinámico según el avance.
FiltrosTareas (propio)	Chips Todas / Pendientes / Completadas con sus conteos: la navegación entre las tres vistas de la lista.

Mascota (propio, react-native-svg)	Personaje vectorial del encabezado que reacciona al avance: duerme con 0%, se pone contento entre 1% y 99% y celebra al 100%.

9. Usabilidad: una interfaz intuitiva y fácil de usar

En este apartado se explica cómo la interfaz resulta intuitiva y fácil de usar, considerando las necesidades y expectativas de las personas que la usan a diario.

- Validación visible: el título es obligatorio. Sin título, el botón Guardar se ve deshabilitado (opacidad 0,45) y se muestra el mensaje «El título es obligatorio: escribe qué tienes que hacer.».
- Mensajes claros y en español: todos los textos de la app están pensados para un usuario no técnico.
- Modo edición evidente: al tocar Editar, el formulario se llena con los datos, su título cambia a «Editar tarea», el botón pasa a «Actualizar tarea» y aparece la opción «Cancelar».
- Confirmación al eliminar: antes de borrar, un Alert pregunta «¿Seguro que quieres eliminar…?» con opciones Cancelar/Eliminar, para evitar borrados por error.
- Estado vacío útil: si no hay tareas, la lista muestra «No tienes tareas todavía» y explica qué hacer a continuación.
- Retroalimentación inmediata: el contador del encabezado se actualiza al agregar, completar o eliminar; la tarea completada se tacha.
- Accesibilidad: accessibilityLabel, accessibilityRole y estados (checked, disabled) en botones, campos y checkbox; textos grandes y zonas táctiles amplias.
- Uso con una mano y en movimiento: botones grandes, estructura vertical simple y teclado que no tapa el formulario (KeyboardAvoidingView).
- Progreso siempre visible: la tarjeta «Tu progreso» muestra la barra animada y el texto «X de Y tareas listas (Z%)», con un mensaje que cambia según el avance.
- Filtros con conteos: los chips Todas / Pendientes / Completadas permiten mirar un solo grupo de tareas; el estado vacío cambia según el filtro (si no queda nada pendiente, el mensaje lo celebra con un trofeo).
- Una mascota que reacciona: el personaje del encabezado duerme sin avance, se pone contento al avanzar y celebra al completar todo. Junto al saludo según la hora, la app se siente viva sin perder claridad; todos los iconos son vectoriales profesionales (Ionicons), siempre acompañados de su palabra.

10. Estructura de archivos del proyecto

El proyecto se organizó con una estructura profesional, por responsabilidad, para que cada archivo haga una sola cosa y sea fácil de ubicar:

```
tareas-expo/
├── App.tsx — Solo raíz de composición: muestra la pantalla (sin lógica)
├── index.ts — Punto de entrada: registra App en Expo (no se toca)
├── app.json — Configuración de la app (nombre «Mis Tareas», ícono, orientación)
├── package.json — Dependencias y scripts (start, android, ios, web, typecheck,
lint)
├── tsconfig.json — Configuración de TypeScript (modo estricto)
├── eslint.config.js / .prettierrc.json / .editorconfig — Calidad y formato
automáticos
├── README.md — Documentación completa del proyecto, paso a paso
├── capturas/ — Capturas de pantalla de la aplicación
└── src/
    ├── pantallas/PantallaTareas.tsx — La pantalla completa (contenedor)
├── componentes/ — Piezas que solo dibujan: Encabezado, Mascota, BarraProgreso,
│   FiltrosTareas, FormularioTarea, TareaItem, EstadoVacio e index.ts (barrel)
    ├── hooks/useTareas.ts — La lógica: agregar, actualizar, eliminar,
completar, contadores
├── servicios/ — Reglas puras: validacionTarea.ts (título obligatorio y
normalización) y filtradoTareas.ts (qué tareas se muestran según el filtro)
├── modelos/tarea.ts — Interfaz Tarea, DatosTarea y tipo FiltroTareas (la forma
de los datos)
├── constantes/ — limites.ts (largos máximos con nombre) y mensajes.ts (saludo
según la hora, mensajes de progreso y de lista vacía)
    ├── utilidades/ids.ts — generarId(): crea el id único de cada tarea
    └── tema/ — Tokens de diseño: colores.ts, espaciado.ts, tipografia.ts e
index.ts
```

Buenas prácticas aplicadas en esta estructura: responsabilidad única por archivo; presentación (componentes) separada de la lógica (hook y servicios) y de los datos (modelo); App.tsx solo como raíz de composición; sin números ni textos mágicos repetidos, sustituidos por constantes y tokens de diseño con nombre; TypeScript estricto; y nombres consistentes (componentes en PascalCase, hooks con prefijo use, servicios y utilidades en camelCase). El detalle práctica por práctica, con el archivo donde se aplica cada una, está en la sección «Estructura profesional y buenas prácticas» del README del proyecto.

11. Funcionalidades implementadas y pruebas

Funcionalidad	Cómo funciona	Prueba realizada
Agregar tarea	Se escribe título (obligatorio) y descripción (opcional) y se toca	Script de lógica: la lista pasa de 3 a 4 tareas y el título queda recortado (trim) —

	«Guardar tarea». La tarea aparece al final de la lista y el formulario se limpia solo.	OK
Editar tarea	«Editar» carga los datos en el formulario; «Actualizar tarea» guarda los cambios y «Cancelar» sale del modo edición.	Script de lógica: título y descripción se actualizan sin tocar el resto — OK
Eliminar tarea	Botón «Eliminar» por tarea + confirmación con Alert; al confirmar, la tarea desaparece de la lista.	Script de lógica: la lista vuelve a 3 y el id ya no existe — OK
Completar tarea	El checkbox marca la tarea: se tacha el texto, el círculo se rellena en verde y cambia el contador.	Script de lógica: completada alterna true/false correctamente — OK
Validación	Sin título no se puede guardar: botón deshabilitado y mensaje de error bajo el campo.	Regla titulo.trim().length > 0 verificada con textos vacío y con espacios — OK
Contadores	El encabezado muestra pendientes y completadas, derivados de la lista con useMemo.	Con las tareas iniciales: 2 pendientes y 1 completada — OK

Además se realizaron las pruebas técnicas: node -v → v24.22.1; npx expo --version → 58.1.15 (SDK 58); npx tsc --noEmit → sin errores; ESLint → sin problemas; npx expo export --platform web → Metro empaquetó 416 módulos sin errores (bundle web de 1,1 MB); y la lógica completa —incluidos los filtros y los mensajes dinámicos— pasó 11/11 casos con las funciones reales del proyecto.

12. Capturas de pantalla de la aplicación en funcionamiento

Las siguientes imágenes reproducen fielmente la aplicación real (mismos colores de src/tema/, mismos textos e iconos de los componentes y mismas tareas de ejemplo de useTareas.ts), en formato de teléfono:



Figura 1. Pantalla principal: encabezado con el saludo según la hora y la mascota contenta, barra de progreso en 33% con su mensaje dinámico, formulario de nueva tarea con iconos profesionales (el botón se ve atenuado hasta escribir un título, por la validación), chips de filtro con sus conteos y las tareas con borde de color según su estado (azul pendiente, verde completada).



Figura 2. Modo edición: el formulario se carga con los datos de la tarea elegida, su título cambia a «Editar tarea», el botón principal pasa a «Actualizar tarea» y aparece la opción «Cancelar» para salir sin guardar cambios.



Figura 3. Todo completado: las tres tareas marcadas y tachadas, los chips muestran Pendientes (0) y Completadas (3), la barra de progreso llega al 100% con el mensaje «¡Todo listo! ¡Completaste todas tus tareas!» y la mascota celebra con los brazos arriba. Es la retroalimentación completa del avance: número, barra, mensaje y personaje cambian con el estado.

12. Conclusiones y proyección

La aplicación cumple su objetivo: se instaló y configuró el entorno de desarrollo (Node.js y Expo), se creó un proyecto de aplicación móvil desde la plantilla en blanco y se implementaron los componentes de la interfaz de usuario, con las funciones de agregar, editar, eliminar y completar tareas, una barra de progreso con filtros y una mascota que reacciona al avance.

El trabajo quedó ordenado por capas —modelo, lógica en un hook, servicios de reglas, componentes presentacionales y tema centralizado—, lo que deja la base preparada para crecer. Los pasos siguientes del proyecto son:

- Almacenamiento local (por ejemplo, AsyncStorage) para que las tareas no se pierdan al cerrar la app.
- Autenticación de usuarios, para que cada persona tenga su propia lista.
- Comunicación con APIs externas, para sincronizar las tareas con un servidor.